

SCF-PDP Programming Exercise Report: Assignment 2

Heuristic Optimization Techniques, WS 2025

Daniel Levin (12433760)

1 General Setup and New Tools

The implementation continues in Python 3.13, building upon the framework established in Assignment 1.

1. The following libraries were added for the second assignment:

- `sklearn.cluster.KMeans` – K-means clustering algorithm for spatial location clustering in enhanced construction heuristics
- `scipy.stats.wilcoxon` – Wilcoxon signed-rank test for statistical comparison of paired algorithm results
- `optuna` – Bayesian optimization framework using Tree-structured Parzen Estimators for automated parameter tuning
- `optuna-dashboard` – Web-based visualization dashboard for monitoring Optuna optimization studies in real-time

2 Extensions from Assignment 1

The results of the algorithms implemented in Assignment 1 were quite poor and the most likely reason for that was the simplicity of the greedy construction which served as the baseline for all other approaches evaluated in Assignment 1. To recall, the original greedy construction always inserted pickup and dropoff locations together based on the proximity of the unserved pickups. Therefore, before starting to work on this assignment I was motivated to try and improve the greedy construction heuristic.

2.1 Idea

I have tried out two enhancements to the greedy construction as well as a hybrid approach combining both enhancements:

- **Flexible Pickup and Dropoff:** Instead of always inserting pickup and dropoff locations together, I have implemented a more flexible approach which evaluates all valid pickup positions and, for each pickup position, all valid dropoff positions. It selects the request with minimum total insertion cost across all position combinations, respecting capacity constraints. This approach is more time consuming so I have to implement multiple optimization such as maintaining a list of carried capacities at every step in the route and optimized based on that capacity constraint validation.
- **Cluster Requests:** Uses K-means clustering on request centroids (midpoint of pickup-dropoff pairs) to pre-distribute requests to vehicles. Each vehicle serves only requests from its assigned cluster.
- **Hybrid:** Combines clustering with flexible dropoff placement within each cluster.

2.2 Results

2.2.1 Greedy vs Flexible Pickup & Dropoff

Figure 1 compares the baseline greedy construction with the flexible pickup and dropoff approach across 30 trains instances of each of the sizes 50, 100, 200, 500, and 1000. As can be seen, the flexible approach consistently outperforms the baseline greedy construction with total average improvement of 56%. This is not surprising because the flexible approach explores a much larger solution space by considering all valid insertion positions for both pickup and dropoff locations. The computational overhead is manageable due to optimizations in capacity constraint checking. Solving 30 instances of size 1000 on 8 processors in parallel takes approximately 10 minutes.

The gap in result improvement grows slightly with instance size, which can be explained by the increasing number of possible insertion positions. Overall, this enhancement significantly boosts solution quality compared to the original greedy construction.

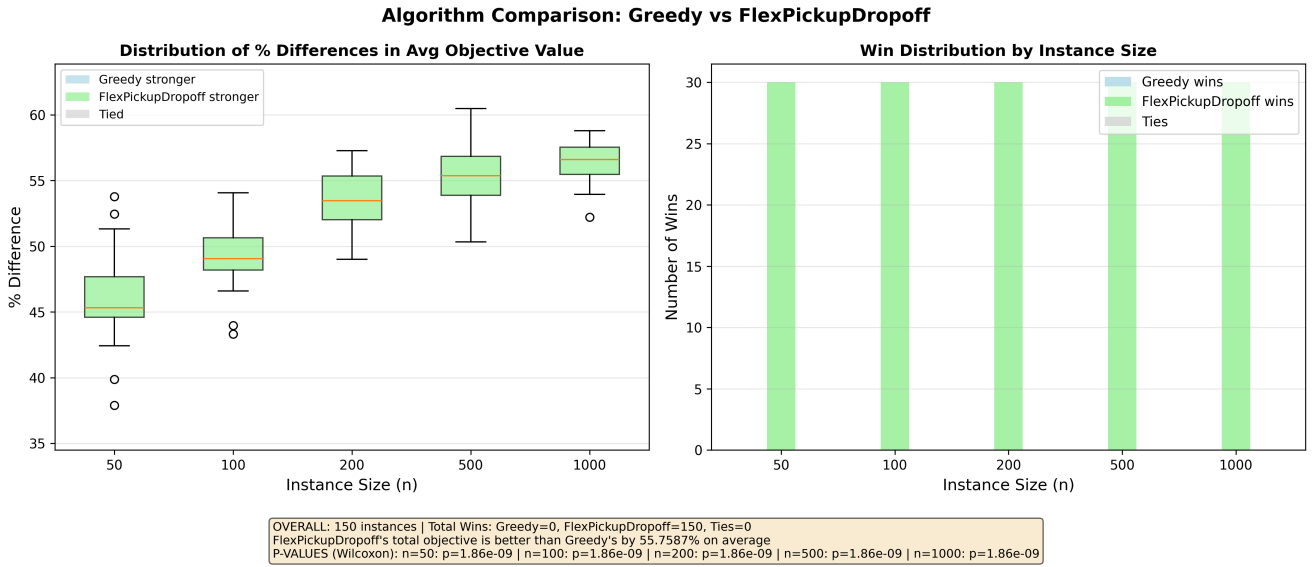


Figure 1: Greedy vs Flexible Pickup & Dropoff

2.2.2 Greedy vs Clustered

Figure 2 compares the baseline greedy construction with the cluster-based approach across instance sizes 50, 100, 200, 500, and 1000. The clustered construction turned out to be less effective than the flexible pickup and dropoff approach, although still outperforming the original greedy approach by 12% on average. The main advantage of clustering is that it reduces the solution space by limiting each vehicle to a subset of requests, that are located close to each other. That means that the clustered approach can also improve performance in larger instances due to divided search space.

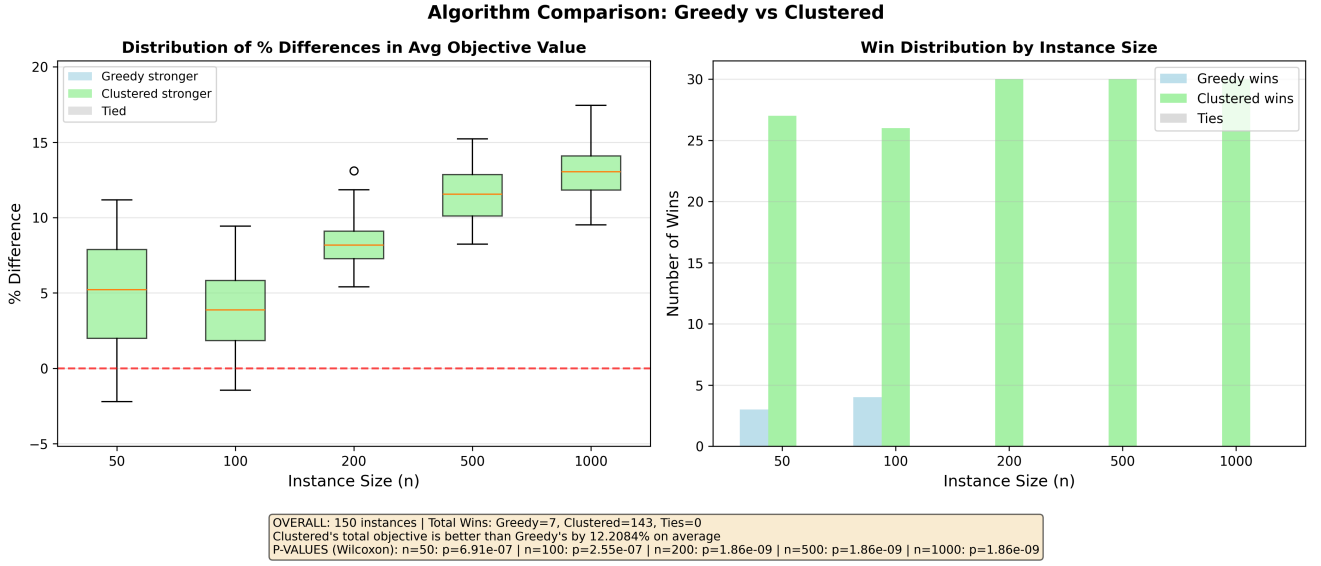


Figure 2: Greedy vs Clustered

2.2.3 Clustered vs Flexible Pickup & Dropoff

Figure 3 compares the cluster-based approach with the flexible pickup and dropoff approach across instance sizes 50, 100, 200, 500, and 1000. As expected from the previous results, the flexible pickup and dropoff approach significantly outperforms the clustered approach by 49% on average. This mainly stems from the fact that the insertion logic used in the clustered approach is the same as in the original greedy construction, which is much more limited in solution exploration. However, these approaches are not mutually exclusive, so if combined we could expect even better results as shown in the next section.

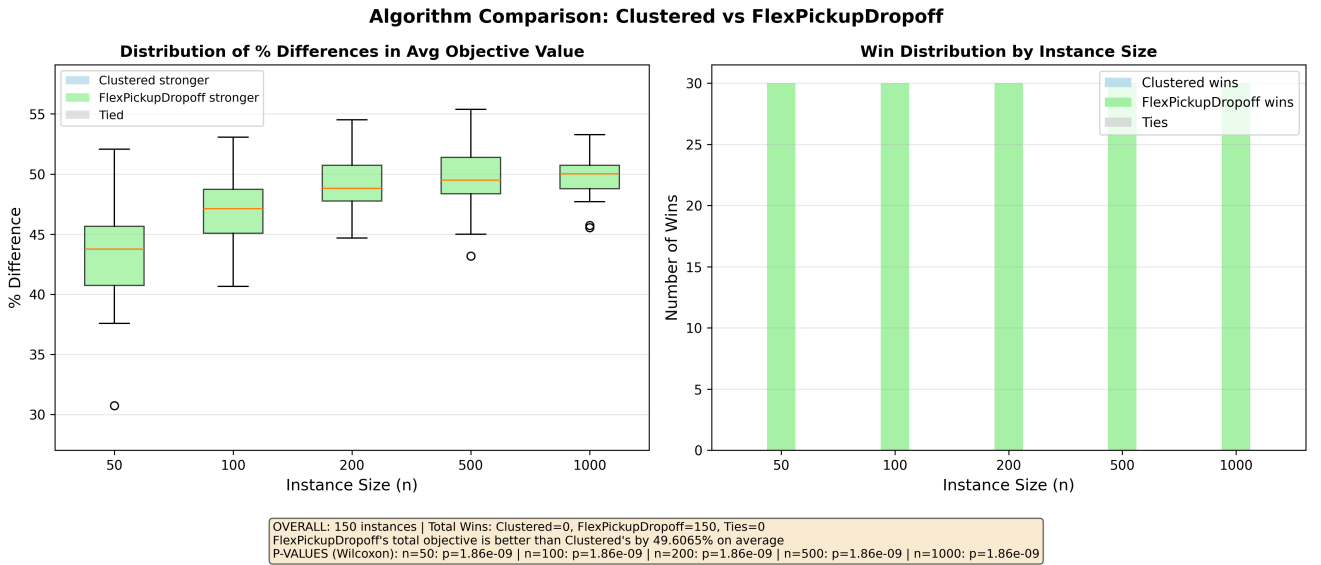


Figure 3: Clustered vs Flexible Pickup & Dropoff

2.2.4 Flexible Pickup & Dropoff vs Hybrid

Finally, the best performing approach (Flexible Pickup & Dropoff) was compared to the hybrid approach that combines clustering and greediness of requests serving provided by the flexible approach. Despite the intuitive expectation that the hybrid will perform better, its quality was worse compared to the flexible approach, especially as the instance size grows. In the hindsight it made sense, because clustering limits the search space significantly without considering the capacity distribution, while the flexible approach

goes over all possible requests assignments greedily. The main benefit of the hybrid approach is reduced runtime explained by smaller search space. As can be seen in Figure 4, for instances of size 50 the Wilcoxon test shows that the H_0 hypothesis cannot be rejected ($p = 0.2801$). However, for larger instances the flexible approach clearly outperforms the hybrid one, with an average objective improvement of 8.49% across all instance sizes.

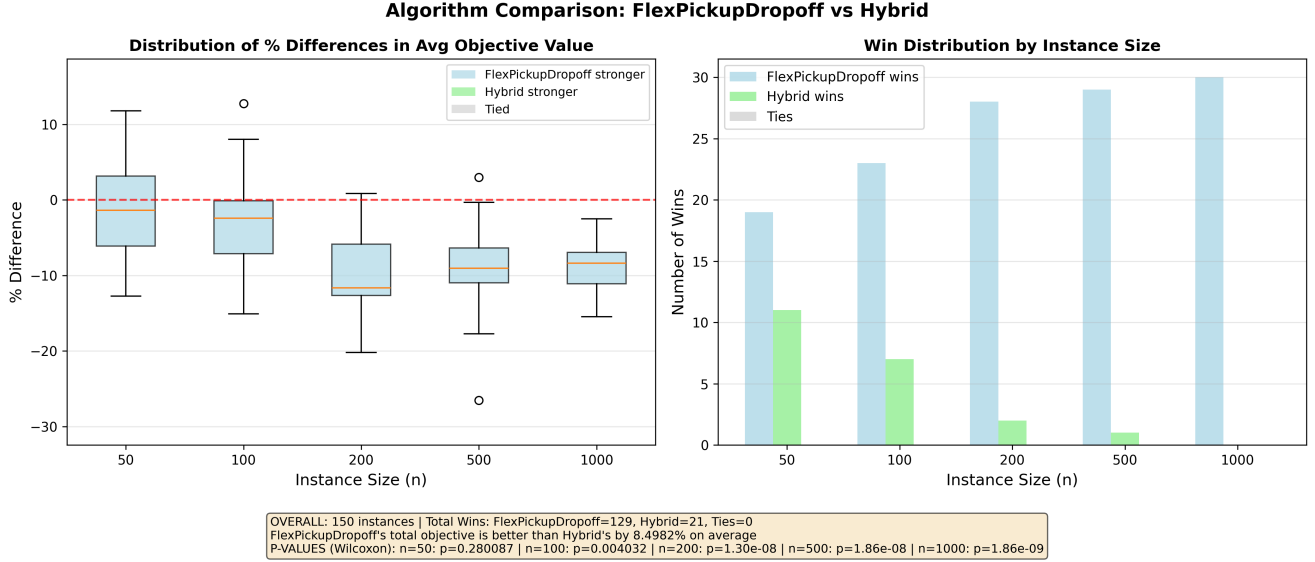


Figure 4: Flexible Pickup & Dropoff vs Hybrid

2.2.5 Runtime Comparison: Flexible vs Hybrid

To showcase the benefit of the hybrid approach in runtime I ran both approaches on 30 instances of size 50, 100, 200, 500 and 1000 and measured average construction runtime as can be seen in Figure 5. The left subplot shows mean runtimes on logarithmic scale, while the right subplot displays speedup ratios with the Flexible approach runtime being the baseline. For instances of size 50, both approaches perform similarly with slightly-negative-to-no speedup, but the advantage of the hybrid approach becomes more pronounced with larger instance sizes, reaching a 13x speedup for instances of size 1000. The overall speedup of 3.59x is computed as the geometric mean of individual speedup ratios rather than arithmetic mean, as geometric mean is more appropriate for multiplicative factors and prevents large ratios from dominating the average. This confirms that clustering effectively reduces the search space, resulting in computational savings despite the solution quality trade-off discussed earlier.

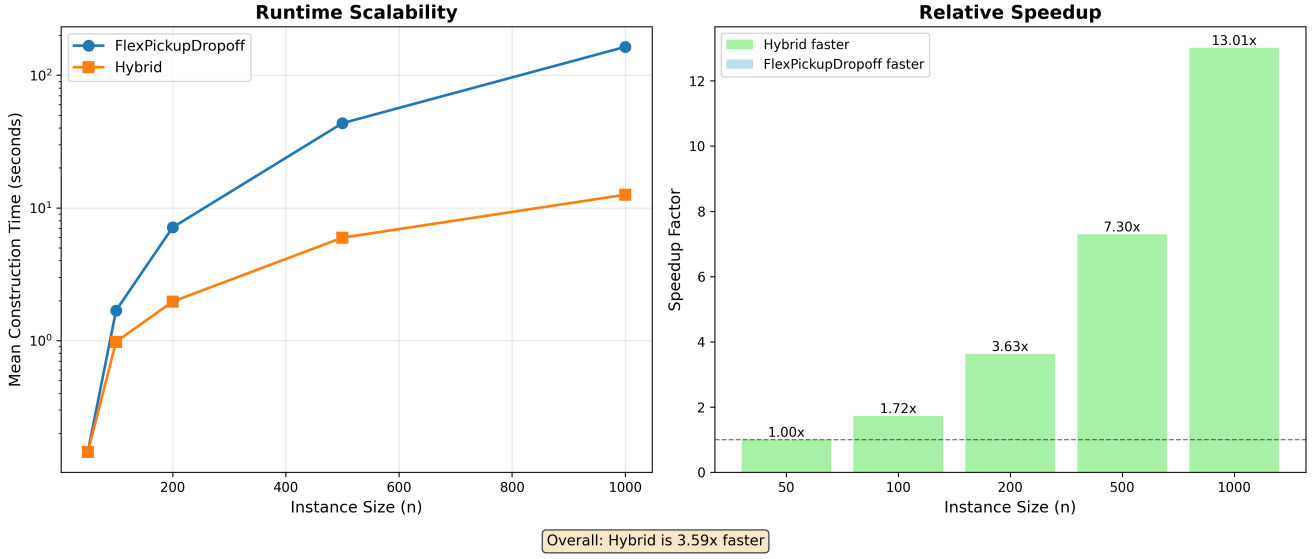


Figure 5: Runtime Comparison: Flexible Pickup & Dropoff vs Hybrid

3 Metaheuristic: ALNS

Adaptive Large Neighborhood Search iteratively destroys and repairs parts of the current solution sampling from sets of operators. Operator selection is based on a wheel roulette choice based on adaptive weights that favor operators discovering better solutions, balancing exploration and exploitation throughout the search. The algorithm combines this with simulated annealing acceptance as suggested in the lecture to escape local optima.

3.1 Operators

The implementation uses 3 destroy operators and 3 repair operators, yielding 9 total combinations chosen for relative simplicity while maintaining good exploration capabilities.

Each destroy operator removes q requests, where q is sampled uniformly from the range

$$[\min_removal_percentage, \max_removal_percentage]$$

of total served requests (default [10%, 40%]).

Destroy Operators:

- **Random Removal:** Removes q randomly selected requests to achieve unbiased exploration of the solution space.
- **Worst Cost Removal:** Removes q requests with highest distance contribution to the objective, targeting expensive assignments for reassignment.
- **Longest Route Removal:** Removes q requests from vehicles with longest routes to improve fairness and balance workload.

Repair Operators:

- **Greedy Repair:** Reinserts removed requests using the flexible construction heuristic, evaluating all pickup-dropoff position combinations and selecting minimum insertion cost.
- **Random Greedy Repair:** Reinserts requests at random feasible positions to introduce diversification.

- **Objective-Aware Repair:** Reinserts requests by minimizing the full objective including fairness penalty, temporarily inserting each request in all feasible positions and evaluating complete solution quality. This repair operator was mainly added to address the fairness measures that are ignored in other operators.

3.2 Adaptive Weight Mechanism

Operators are scored based on solution quality: to reward operators that discover new best solutions, they're awarded 10 points, while accepted solutions award 1 point. Rejected solutions are implicitly awarded 0 points. Weights are updated every 100 iterations using the adaptive formula

$$\rho_i \leftarrow \rho_i \cdot (1 - \gamma) + \gamma \cdot \frac{s_i}{a_i}$$

where ρ_i is the weight, γ is the reaction factor, s_i is the total score, and a_i is the number of applications. This mechanism rewards operators that consistently find improvements while maintaining exploration thanks to non-zero initial weights. Operator selection uses roulette wheel selection with probabilities proportional to current weights.

3.3 Parameters

Table 1: ALNS Parameters

Parameter	Description	Default Value
max_iterations	Maximum iterations	10000
max_time_seconds	Maximum runtime	300s (5 min)
max_iterations_without_improvement	Early stopping threshold	1000
weight_update_period	Weight update frequency	100
reaction_factor (γ)	Adaptation speed	0.1
min_removal_percentage	Minimum requests to remove	10%
max_removal_percentage	Maximum requests to remove	40%
initial_temperature	Initial SA temperature	100.0
cooling_rate	Temperature decay	0.99
score_new_best	Reward for new best	10.0
score_accepted	Reward for acceptance	1.0

4 Fairness Measure Investigation

Three fairness measures were evaluated. For each of them values closer to 1 indicate more balanced workload distribution which makes them comparable.

- **Jain Fairness Index:** $J = \frac{(\sum_{k \in K} d(R_k))^2}{n_K \sum_{k \in K} d(R_k)^2}$ where $d(R_k)$ is the distance of route k .
- **Max-Min Fairness:** $M = \frac{\min_{k \in K} d(R_k)}{\max_{k \in K} d(R_k)}$ measuring the ratio of the gap between shortest and longest routes.
- **Gini Coefficient (Inverted):** $G = 1 - \frac{\sum_{k' \in K} \sum_{k'' \in K} |d(R_{k'}) - d(R_{k''})|}{2n_K \sum_{k' \in K} d(R_{k'})}$.

4.1 Experimental Setup

After having considered multiple setups, I have decided to use a single instance of size 500 with 10 vehicles as this is the smallest instance with sensible number of vehicles where we can actually compare the distribution of distances and stops across these vehicles. For ALNS 60-second timeout was used per run. Three main experiments have been conducted:

1. **Greedy vs ALNS:** Flexible greedy construction heuristic (blind to fairness by design) vs ALNS with all destroy and repair operators (3×3).
2. **ALNS Objective-Aware Repair:** ALNS with all operators vs ALNS without ObjectiveAware repair vs ALNS with only ObjectiveAware repair.
3. **ALNS Longest-Route Destroy:** ALNS with all operators vs ALNS without LongestRoute destroy vs ALNS with only LongestRoute destroy.

Each experiment evaluates all three fairness measures (Jain, Max-Min, Gini). Results were analyzed across objective value composition to capture how much the fairness measure contributes to the total objective value, route distance distributions to compare how different measures affect the distribution of routes, and number of stops per route to examine another angle of fairness.

4.2 Greedy vs ALNS

This experiment compares the construction heuristic baseline against ALNS to evaluate the metaheuristic’s ability to improve solutions for different fairness measures. Figure 6 shows objective values and composition, while Figure 7 presents fairness index values and route distributions.

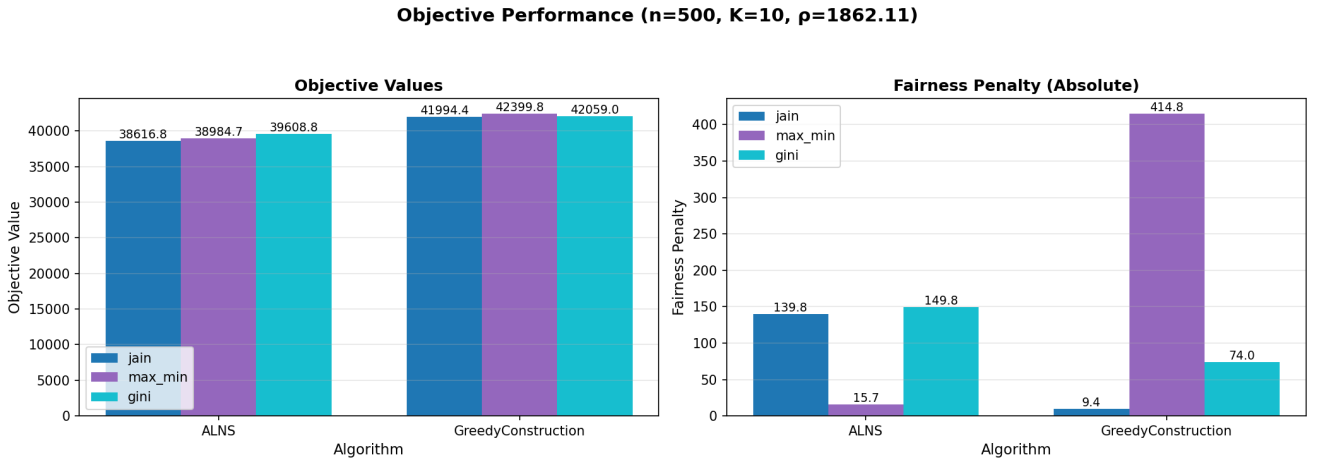


Figure 6: Algorithm Comparison: Objective Performance

The left subplot of Figure 6 shows that ALNS produces best results for **jain** index, slightly worse for **max_min**, and worst for **gini** index. The right subplot reveals that minimal fairness penalty (contribution of fairness index to total objective) is paid for **max_min** in contrast to the **Greedy** approach, proving that diverse destroy and repair operators embedded in ALNS can capture and learn different objectives. The **LongestRouteRemovalOperator** helps the algorithm identify long routes invisible to **Greedy** construction and redistribute requests between vehicles, demonstrating that destroy and repair operator choice is the main factor affecting fairness performance in ALNS. For the **Greedy** approach, performance varies across fairness measures due to its deterministic and fairness-agnostic construction—requests are distributed evenly between vehicles, which explains why **jain** and **gini** result in lowest fairness penalty, while **max_min** sees an increase because even request distribution does not guarantee fairness in distance lengths per vehicle, potentially leaving distant outlier vehicles accidentally assigned geographically spread requests.

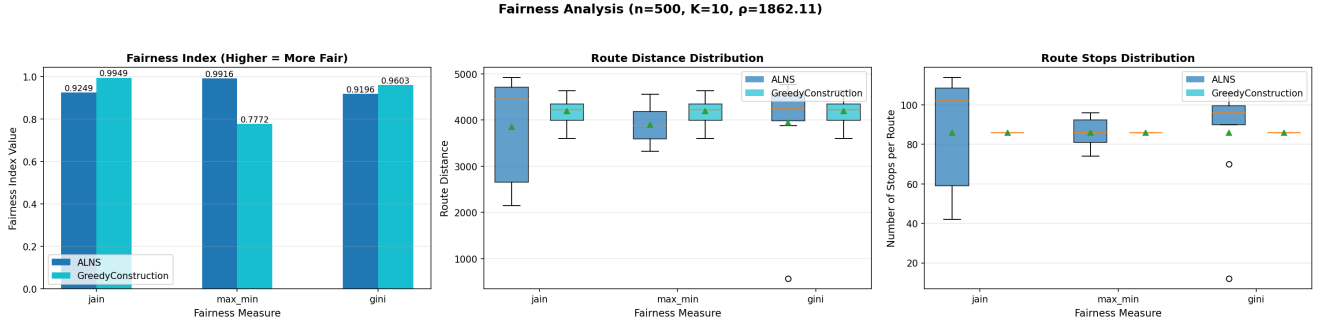


Figure 7: Algorithm Comparison: Fairness Analysis

Figure 7 reveals the actual fairness of achieved solutions. The Fairness Index subplot shows that Greedy construction optimizes well for balanced indexes like `jain` and `gini`, outputting fairness values around 0.9, while for the unbalanced `max_min` it achieves only 0.77. In contrast, ALNS leverages LongestRouteRemovalOperator to reach 0.9916 even for this challenging index, demonstrating ALNS’s universality despite its dependence on operator choice. The middle and right subplots show Greedy generating deterministic routes with constant stops per vehicle, while ALNS produces more diverse routes, especially for `jain` index. For ALNS, mean values fall below median for both route distances and stops, indicating workload distribution skewed toward few very short routes—this skew is most pronounced for `gini` with one outlier below 1000 distance and 20 stops versus medians above 4000 and 90 respectively. With 10 vehicles serving 500 requests (average ~ 50 requests per route), these extreme outlier routes (20 vs 90 stops) suggest `gini` optimization may sacrifice some vehicles entirely to achieve better overall pairwise differences, as the `gini` coefficient’s sensitivity to all pairwise differences amplifies the impact of extreme values in the adaptive weight mechanism. In contrast, `max_min` shows near-perfect mean-median alignment because this index only considers shortest and longest routes, causing the algorithm to minimize their difference and squeeze remaining routes densely between them.

4.3 ALNS Objective-Aware Repair

This experiment isolates the impact of the `ObjectiveAwareRepairOperator`, which explicitly minimizes the full objective including fairness penalty during reinsertion. Thanks to this experiment I found a bug in fairness calculation for the `ObjectiveAware` repair operator, which was fixed before conducting the final experiment. Three ALNS configurations were tested: with all operators, without `ObjectiveAware`, and with only `ObjectiveAware`. Figure 8 and Figure 9 show the results.

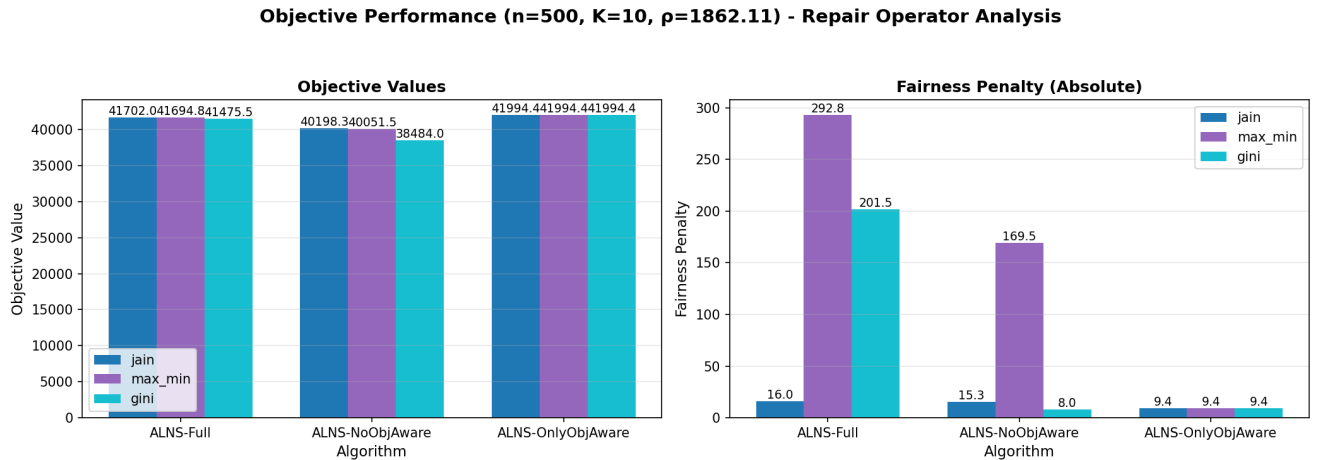


Figure 8: ALNS Objective-Aware Repair Impact: Objective Performance

Figure 8 shows that the version without `ObjectiveAware` (ALNS-NoObjAware) provided the best results. This is counterintuitive—debugging the `ObjectiveAware` operator revealed no bugs, suggesting

the negative impact stems from slower runtime due to frequent objective recalculation. For **ALNS**, when one operator is slower than others, it reduces the number of iterations the entire algorithm can complete within the time limit, and my implementation has no punishment mechanism to account for operator runtime. The right subplot confirms this problem, as **ALNS-OnlyObjAware** produces identical values for all three fairness indexes in both total objective and fairness penalty. If indeed bug-free, this suggests **ALNS** reaches local optima it cannot escape, which is more likely with a single repair operator. Additionally, **max_min** resulted in the highest fairness penalty, yet this didn't worsen overall objective value, indicating that **ALNS** and the operators implicitly found good balance between distances and fairness. Meanwhile, **jain** was optimized well in both cases but with higher total objective, suggesting the operators implicitly favor fair distribution over total distance minimization, which constitutes the larger portion of objective values.

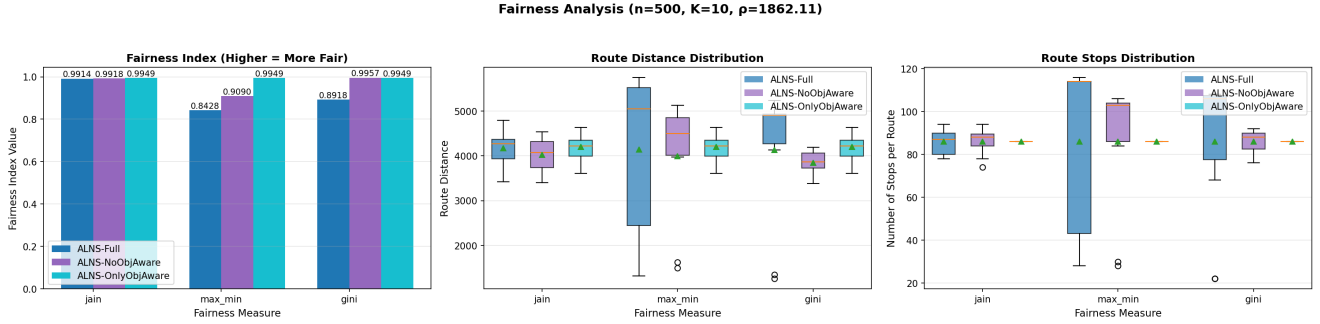


Figure 9: ALNS Objective-Aware Repair Impact: Fairness Analysis

Figure 9 confirms insights from Figure 8—diverse operators (**ALNS-Full**) achieve better total objective by sacrificing fairness index significance. This is evident for **max_min** and **gini**, where fairness index is lowest when using all operators. Route distances and stops are most imbalanced with few very short routes for **ALNS-Full** and **ALNS-NoObjAware** variants, proving that **ObjectiveAware** does what it's designed to do: optimize the objective function as a whole. The **ALNS-OnlyObjAware** variant produces fixed stops per route (~ 85) with worse total objective as shown in Figure 8. The **max_min** showed highest variation in distances and stops for **ALNS** variants with multiple operators, indicating **ALNS** didn't capture objective fluctuations caused by this fairness index, again supported by fairness being negligible compared to distance in total objective calculation. This is not the case for **jain**, likely due to its quadratic nature that punishes imbalanced routes more strongly.

4.4 ALNS Longest-Route Destroy

This experiment evaluates the **LongestRouteRemovalOperator**, which targets route balancing by removing requests from longest routes. Three **ALNS** configurations were tested: with all operators, without **LongestRoute**, and with only **LongestRoute**. Figure 10 and Figure 11 show the results.

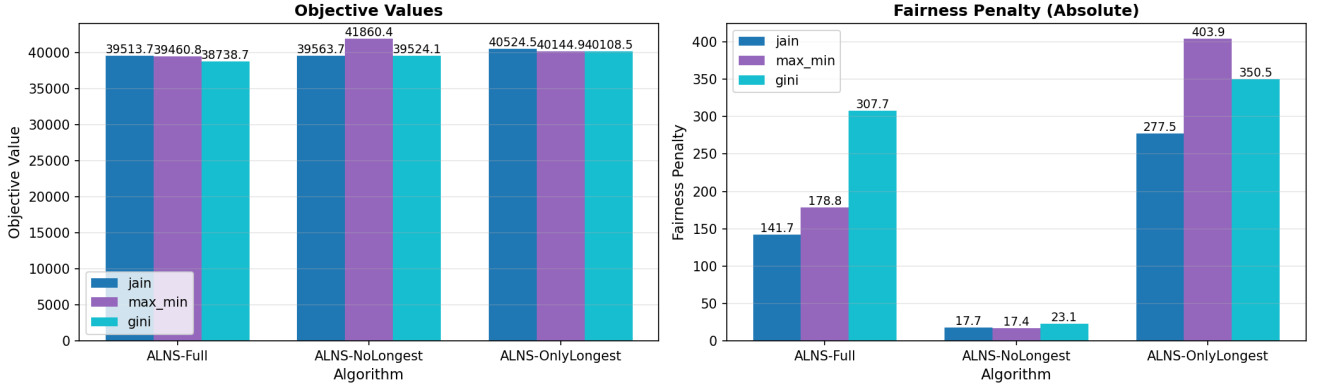
Objective Performance (n=500, K=10, $\rho=1862.11$) - Destroy Operator Analysis

Figure 10: ALNS Longest-Route Destroy Impact: Objective Performance

The results reveal a counterintuitive finding: **ALNS-NoLongest** (without the fairness-targeting operator) produces the most fair solutions across all measures ($\text{jain}=0.9905$, $\text{max_min}=0.9907$, $\text{gini}=0.9876$), while **ALNS-OnlyLongest** produces the least fair solutions ($\text{jain}=0.8510$, $\text{max_min}=0.7831$, $\text{gini}=0.8118$). This contradicts the intuition that explicitly targeting fairness should improve it. Several explanations are possible: (1) operator diversity enables more effective search—the combination of **Random** and **WorstCost** operators may naturally balance routes while **LongestRoute** alone is too myopic; (2) **LongestRoute** may be overly aggressive, destroying good route structures that repair operators struggle to rebuild while maintaining balance; (3) single-operator variants limit exploration and prevent the adaptive weight mechanism from learning effective strategies, potentially trapping ALNS in local optima as observed with **ObjectiveAware**; (4) the standard deviation of stops per route confirms this—**NoLongest** shows remarkably low variation (3.58-6.75) indicating naturally balanced solutions, while **OnlyLongest** shows high variation (39.14-48.29) suggesting disruptive behavior. This can be verified by analyzing operator selection frequencies over iterations, plotting convergence curves to detect early stagnation, examining solution diversity throughout search, testing different removal percentages for **LongestRoute**, and comparing iteration counts between variants to rule out runtime effects. The higher objective values for **NoLongest** suggest it prioritizes fairness at the expense of total distance, yet this tradeoff still yields better overall objective than **OnlyLongest** for most fairness measures, further supporting the hypothesis that operators combination matters more than individual operator design.

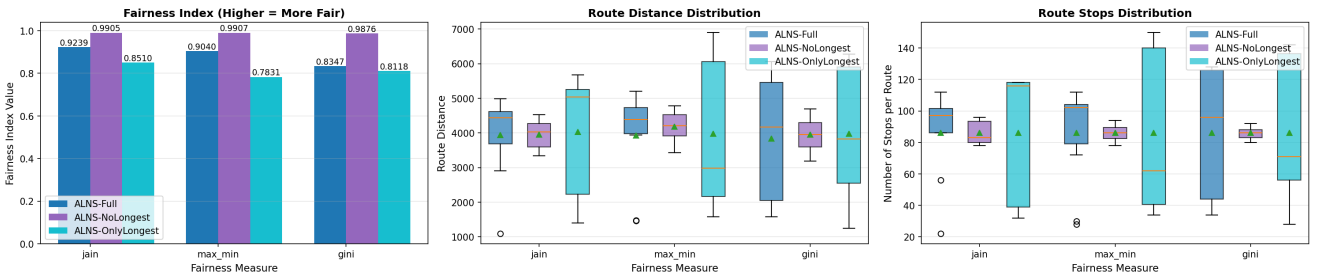
Fairness Analysis (n=500, K=10, $\rho=1862.11$)

Figure 11: ALNS Longest-Route Destroy Impact: Fairness Analysis

Figure 11 reveals how different fairness indexes respond to operator configurations. For **ALNS-NoLongest**, all three indexes achieve remarkably similar high fairness values (0.99+) with minimal route variation, suggesting that **Random** and **WorstCost** operators naturally produce balanced solutions regardless of fairness measure. In contrast, **ALNS-OnlyLongest** exhibits strong sensitivity to fairness measure choice—**max_min** suffers most (0.7831) with highest route variation ($\text{std}=48.29$), while **jain** performs relatively better (0.8510, $\text{std}=39.14$), likely because **jain**'s quadratic penalty on variance makes the adaptive weight

mechanism respond more strongly to imbalanced routes. For **ALNS-Full**, **gini** achieves the best total objective (38738) but at the cost of lowest fairness (0.8347), while **max_min** produces more balanced routes (0.9040) with higher objective (39460), demonstrating that different fairness measures lead to fundamentally different solution structures even with identical operators. The consistently low standard deviation for **ALNS-NoLongest** (3.58-6.75 stops) versus **ALNS-OnlyLongest** (39.14-48.29 stops) across all fairness measures confirms that operator diversity, not explicit fairness targeting, is the primary driver of balanced solutions. This suggests the choice of fairness measure matters most when operator diversity is limited—**ALNS-Full** and **ALNS-NoLongest** adapt well to any fairness measure, while **ALNS-OnlyLongest** struggles particularly with **max_min** due to its focus solely on extremes without the exploration capabilities that other operators provide.

4.5 Conclusion

The **gini** coefficient showed less sensitivity to small numbers of outliers compared to **jain**, which is expected due to **gini**'s linear ratio as a function of route differences, while **jain** focuses strongly on variance by squaring distances. The **max_min** made **ALNS** produce the most balanced solutions by grouping all routes between the shortest and longest routes. However, for other approaches (as shown for **Greedy**), this fairness index might cause worse results. Across all three experiments (**Greedy** vs **ALNS**, **ObjectiveAware Repair**, **LongestRoute Destroy**), a consistent pattern emerges: operator diversity is critical—variants with multiple operators consistently outperformed single-operator configurations, even when that single operator explicitly targeted fairness or objective optimization. This suggests that **ALNS**'s strength lies not in individual operator sophistication, but in the diverse effect that enables escaping from local optima and adaptation to different problem characteristics. Bottom line: more advanced metaheuristics such as **ALNS** are more universal, as they capture the sensitivity of the objective function much better than simpler **Greedy** approaches. However, they're hard to fine-tune and as shown the choice and combination of operators has a significant impact on the algorithm's ability to balance different components of the objective function.

5 Automated Parameter Tuning

For automated parameter tuning, I decided to use **Optuna**¹, a Python-based Bayesian optimization framework using Tree-structured Parzen Estimators (TPE). Optuna was selected over alternative advanced frameworks suggested in the lecture such as SMAC3 due to its simpler API, straightforward parallelization, and suitability for our parameter space size (8 tunable parameters). While both frameworks offer comparable optimization efficiency, Optuna's minimal integration overhead and native support for parallel evaluation through a simple **n_jobs** parameter made it more practical given time constraints. The TPE algorithm models the distribution of good and bad configurations separately, using this knowledge to suggest promising parameter combinations that balance exploration of uncertain regions with exploitation of known good areas. This approach is more efficient than grid or random search, as it learns from previous evaluations to guide the search toward optimal configurations, typically requiring fewer trials to converge to high-quality parameter settings. Additionally, Optuna offers out-of-the-box visualizations that can be stored in a sqlite database that I will be using in this section of the report. A secondary reason for choosing Optuna was a recommendation I received from another student who was using Optuna for fine-tuning an algorithm for a heuristics optimization problem in her Bachelor's Thesis.

5.1 Tuning Setup

The tuning process used 5 training instances per instance size. For instances of size 50 the timeout of 30 seconds was used and the total runtime of the fine-tuning was around 18 minutes which proved that the Optuna achieves a good parallelization given that it tried 50 trials (configurations) for 5 training instances → 250 **ALNS** runs each limited to 30 seconds → 125 minutes sequentially. There are 8 cpu

¹<https://optuna.org/>

cores on my machine so the maximal concurrency fold factor is 8 meaning that optimally parallelized fine-tuning would have run for 15.6 minutes. Together with the overhead of initialization, TPE sampler updated and processes spawning and management 18 minutes is a good runtime.

For instances of size 100, 200, 500 and 1000 I used the timeout of 60 seconds per instance to let it explore the larger search space better. Each of these completed in around 40 minutes. Optuna was executed with 50 trials using TPE sampler with seed 42 for reproducibility. The objective function minimized the average objective value across all training instances. Eight ALNS parameters were tuned while three parameters (`max_iterations=10000`, `max_time_seconds=60`, `max_iterations_without_improvement=1000`) remained fixed to ensure consistent termination criteria.

Table 2: ALNS Tuning Parameter Search Space

Parameter	Type	Search Range	Default
<code>weight_update_period</code>	Integer	[50, 500]	100
<code>reaction_factor</code> (γ)	Float	[0.01, 0.5]	0.1
<code>min_removal_percentage</code>	Float	[0.05, 0.2]	0.1
<code>max_removal_percentage</code>	Float	[0.3, 0.6]	0.4
<code>initial_temperature</code>	Float	[50.0, 500.0]	100.0
<code>cooling_rate</code>	Float	[0.95, 0.9999]	0.99
<code>score_new_best</code>	Float	[5.0, 20.0]	10.0
<code>score_accepted</code>	Float	[0.5, 2.0]	1.0

5.2 ALNS Tuning Results

The tuning process explored 50 different parameter configurations for each instance size, using TPE’s Bayesian optimization to navigate the search space. For instance size 50, the best configuration achieved an average objective value of **1529.0** across the 5 training instances. Table 3 presents the optimized parameters compared to default values.

Table 3: Tuned ALNS Parameters for Instance Size n=50

Parameter	Default	Tuned Value	Change
<code>weight_update_period</code>	100	275	+175%
<code>reaction_factor</code> (γ)	0.1	0.478	+378%
<code>min_removal_percentage</code>	0.1	0.094	-6%
<code>max_removal_percentage</code>	0.4	0.530	+33%
<code>initial_temperature</code>	100.0	437.5	+338%
<code>cooling_rate</code>	0.99	0.956	-3%
<code>score_new_best</code>	10.0	13.4	+34%
<code>score_accepted</code>	1.0	1.59	+59%
Best Objective	1529.0		

The tuning results reveal several key insights. The significantly increased `reaction_factor` (0.478 vs 0.1) suggests faster adaptation to operator performance is beneficial for smaller instances, allowing the

algorithm to quickly identify and exploit successful operators. The much larger `weight_update_period` (275 vs 100) indicates that less frequent weight updates work better, possibly because more iterations are needed to accumulate meaningful statistics before adjusting operator weights. The substantially higher `initial_temperature` (437.5 vs 100.0) combined with faster `cooling_rate` (0.956 vs 0.99) creates a steeper temperature decline, enabling early exploration followed by convergence and local exploitation. The increased scoring rewards (`score_new_best`=13.4, `score_accepted`=1.59) amplify the distinction between finding new best solutions and accepting improvements, encouraging operators that discover global improvements rather than incremental gains. The removal percentage range widened slightly (9.4%-53% vs 10%-40%), giving operators more flexibility in neighborhood size. Overall, the tuned configuration favors faster adaptation, more aggressive exploration early on, and stronger differentiation of operator performance compared to conservative defaults.

5.2.1 Tuning Results Across Instance Sizes

Table 4, Table 5, and Table 6 present the tuned parameters for instance sizes 100, 200, and 500 respectively.

Table 4: Tuned ALNS Parameters for Instance Size n=100

Parameter	Default	Tuned Value	Change
<code>weight_update_period</code>	100	407	+307%
<code>reaction_factor</code> (γ)	0.1	0.318	+218%
<code>min_removal_percentage</code>	0.1	0.118	+18%
<code>max_removal_percentage</code>	0.4	0.437	+9%
<code>initial_temperature</code>	100.0	267.0	+167%
<code>cooling_rate</code>	0.99	0.975	-1.5%
<code>score_new_best</code>	10.0	9.97	-0.3%
<code>score_accepted</code>	1.0	0.776	-22%
Best Objective	4384.6		

Table 5: Tuned ALNS Parameters for Instance Size n=200

Parameter	Default	Tuned Value	Change
<code>weight_update_period</code>	100	97	-3%
<code>reaction_factor</code> (γ)	0.1	0.284	+184%
<code>min_removal_percentage</code>	0.1	0.073	-27%
<code>max_removal_percentage</code>	0.4	0.362	-10%
<code>initial_temperature</code>	100.0	422.5	+323%
<code>cooling_rate</code>	0.99	0.976	-1.4%
<code>score_new_best</code>	10.0	14.53	+45%
<code>score_accepted</code>	1.0	0.847	-15%
Best Objective	11194.5		

Table 6: Tuned ALNS Parameters for Instance Size n=500

Parameter	Default	Tuned Value	Change
<code>weight_update_period</code>	100	55	-45%
<code>reaction_factor</code> (γ)	0.1	0.210	+110%
<code>min_removal_percentage</code>	0.1	0.065	-35%
<code>max_removal_percentage</code>	0.4	0.511	+28%
<code>initial_temperature</code>	100.0	178.8	+79%
<code>cooling_rate</code>	0.99	0.999	+0.9%
<code>score_new_best</code>	10.0	16.29	+63%
<code>score_accepted</code>	1.0	1.06	+6%
Best Objective	42939.1		

Comparing tuned parameters across instance sizes reveals certain trends. The `weight_update_period` decreases as instances grow ($275 \rightarrow 407 \rightarrow 97 \rightarrow 55$), suggesting larger instances benefit from more frequent weight adjustments to adapt quickly to the expanding search space. The `reaction_factor` consistently decreases with instance size ($0.478 \rightarrow 0.318 \rightarrow 0.284 \rightarrow 0.210$), indicating slower, more conservative adaptation becomes preferable as problem complexity increases or alternatively being a reflection of the decreasing `weight_update_period` which balances out slower adaptation. `cooling_rate` approaches 1.0 for larger instances ($0.956 \rightarrow 0.975 \rightarrow 0.976 \rightarrow 0.999$), creating much slower temperature decline—for n=500, the higher cooling rate effectively maintains high temperature throughout the search, prioritizing exploration over exploitation which makes more sense for larger instances with larger search space. The `score_new_best` reward increases with instance size ($13.4 \rightarrow 9.97 \rightarrow 14.53 \rightarrow 16.29$), amplifying the incentive for operators discovering global improvements in larger search spaces. Figure 12 visualizes these parameter evolution trends across instance sizes.

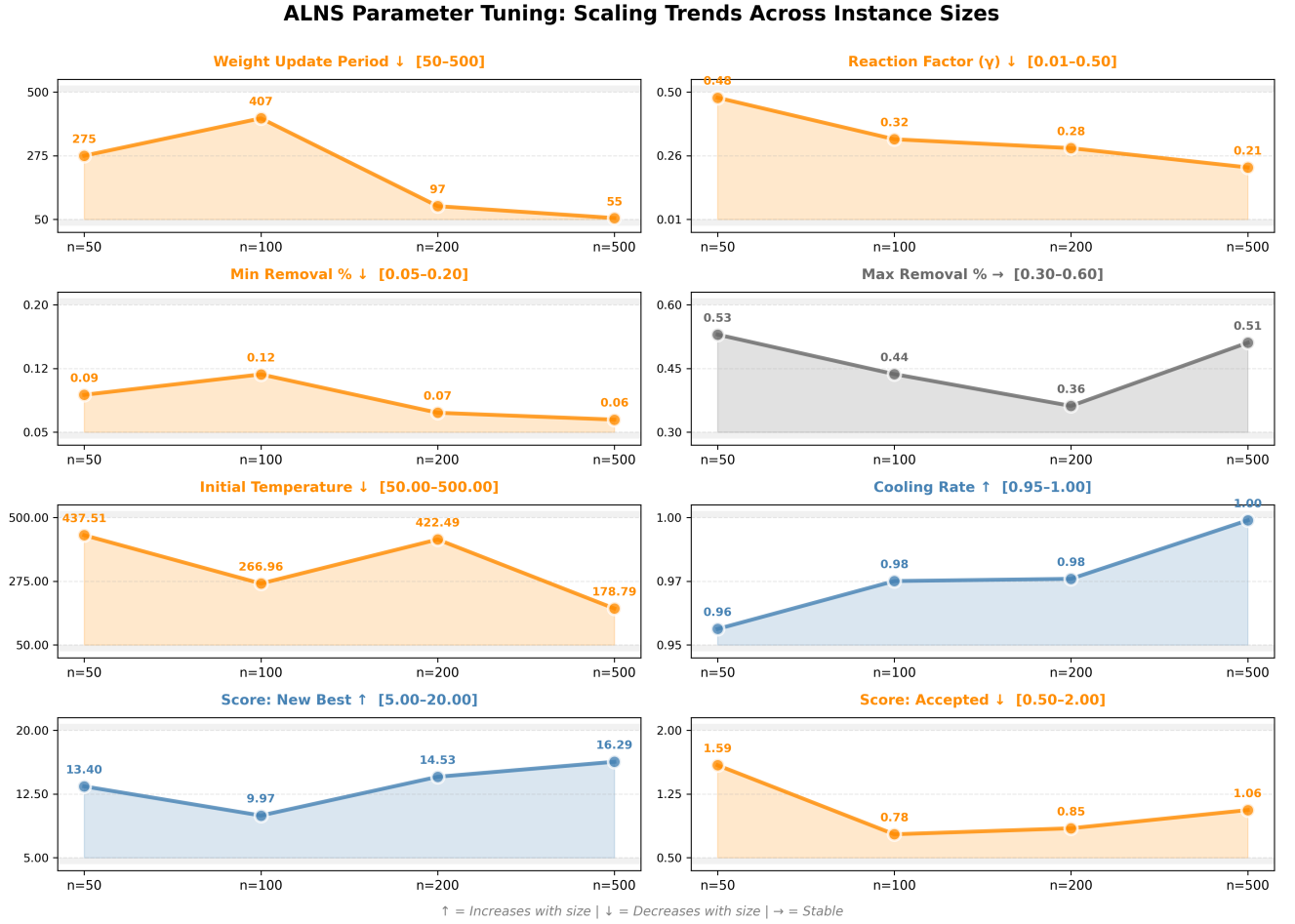


Figure 12: Evolution of Tuned ALNS Parameters Across Instance Sizes

5.2.2 Optimization Convergence and Parameter Importance

The following figures visualize the Optuna optimization process across different instance sizes, showing how the objective value improves over trials and which parameters have the most significant impact on solution quality.

Instance Size n=100:

Figure 13 shows the optimization convergence over 50 trials for instance size 100. The Bayesian optimization finds several configurations that improve the objective value, though plateaus appear throughout the 50 trials. Worse objectives keep occurring as tuning progresses, suggesting Optuna hasn't converged to the best configuration space within 50 trials.

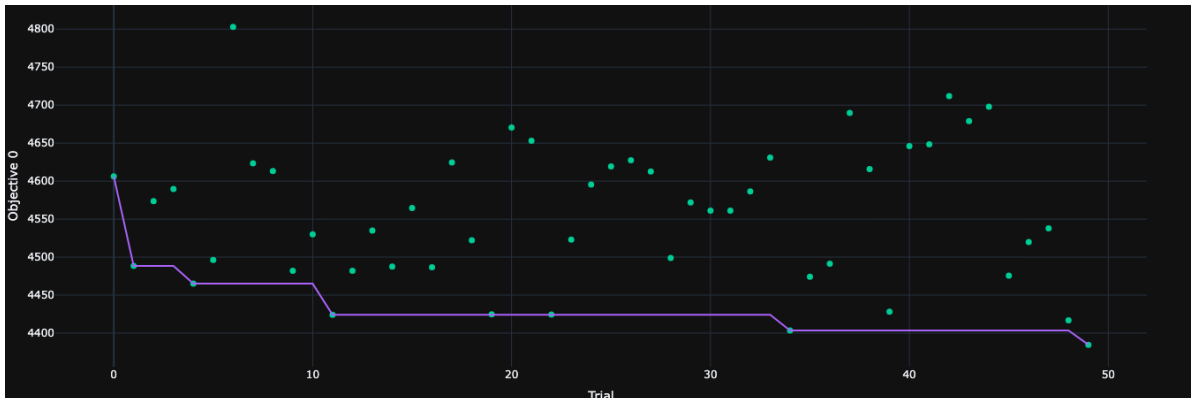


Figure 13: ALNS Tuning Convergence for n=100 (50 trials, 60s budget)

Figure 14 displays parameter importance, indicating which parameters most strongly influence objective value for this instance size. `weight_update_period` dominates parameter importance with 0.49. This parameter doesn't show consistent importance across instance sizes, suggesting Optuna spotted a strong correlation early in this run and explored it further with marginal improvements. It's expected that `weight_update_period` influences ALNS strongly since it controls how often operator weights are adjusted.

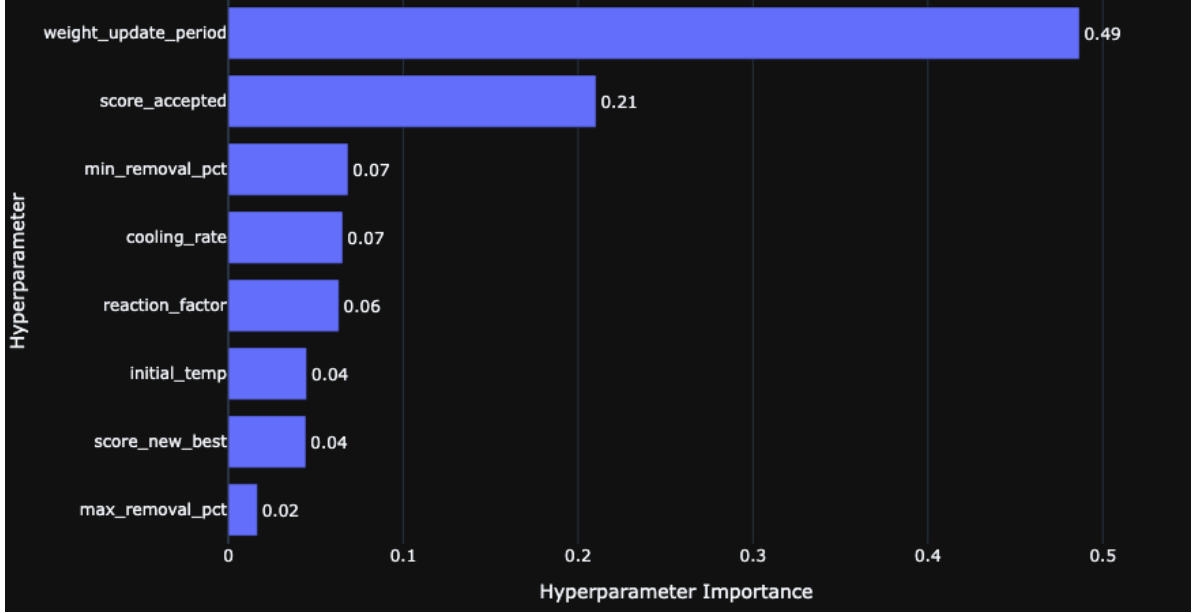


Figure 14: ALNS Parameter Importance for n=100

Instance Size n=200:

Figure 15 shows the optimization convergence for instance size 200. The initial plateau is significant for n=200, meaning Optuna cannot force ALNS to escape local optima through parameter sampling alone. A slight improvement appears toward the end but nothing substantial.

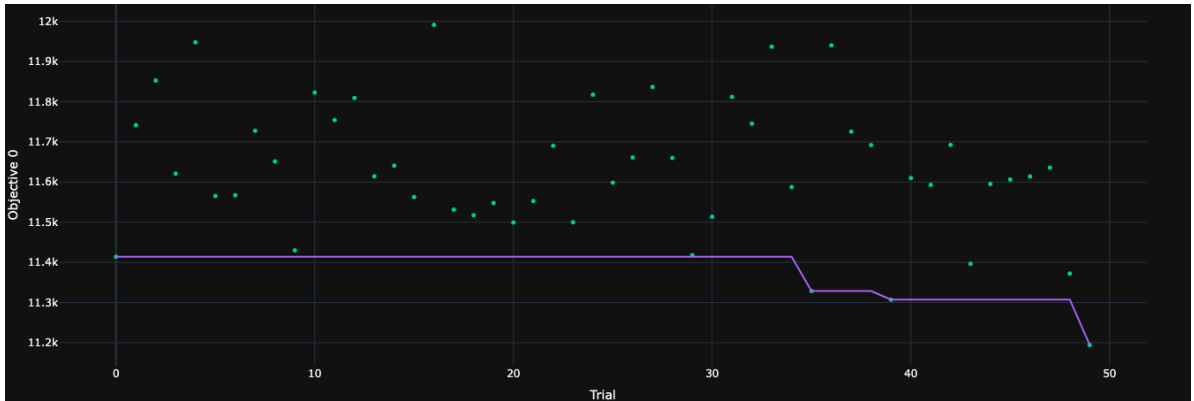


Figure 15: ALNS Tuning Convergence for n=200 (50 trials, 60s budget)

Figure 16 shows parameter importance for instance size 200. Cooling rate accounts for 0.33 of the importance, but overall the parameters are distributed relatively evenly, suggesting Optuna hasn't identified dominant parameters for this size.

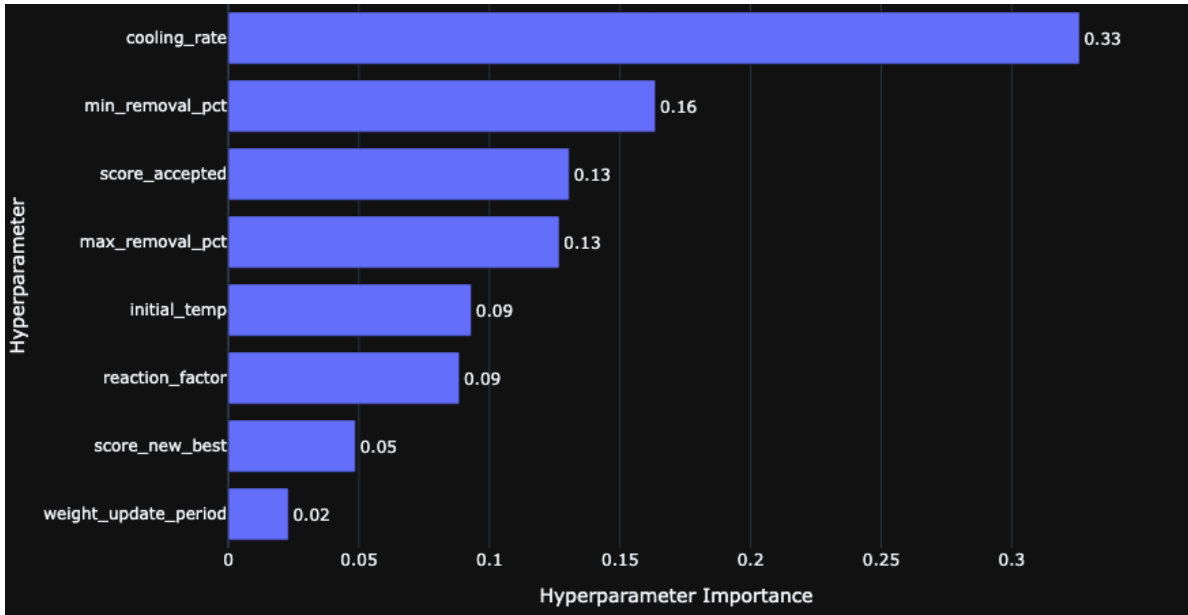


Figure 16: ALNS Parameter Importance for n=200

Instance Size n=500:

Figure 17 shows the optimization convergence for instance size 500. TPE finds improvements despite the relatively low trial count (50), with notable gains around trials 5 and 23. Without a clear progression pattern, Optuna appears not to have learned which configurations work best.

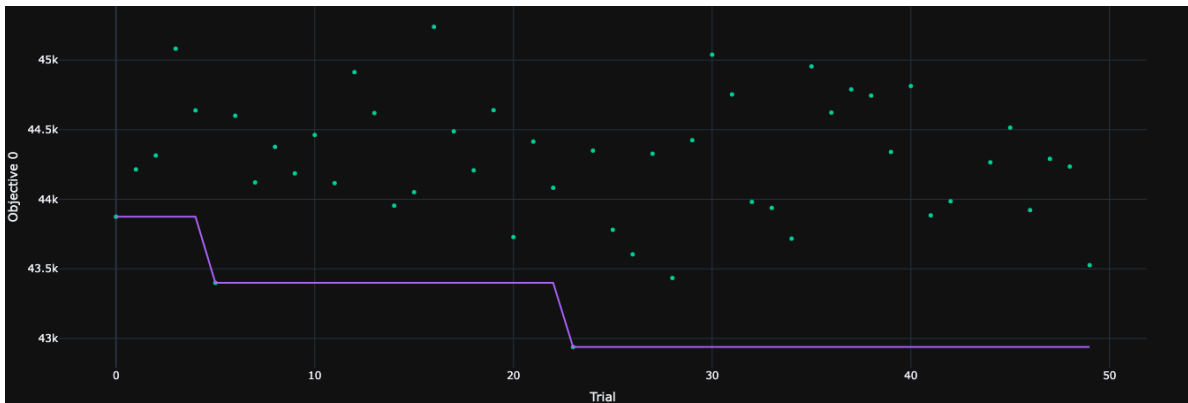


Figure 17: ALNS Tuning Convergence for n=500 (50 trials, 60s budget)

Figure 18 shows parameter importance for instance size 500. Parameter importance remains fairly distributed, though cooling rate gains prominence. Larger search spaces likely benefit from slower cooling to allow thorough exploration.

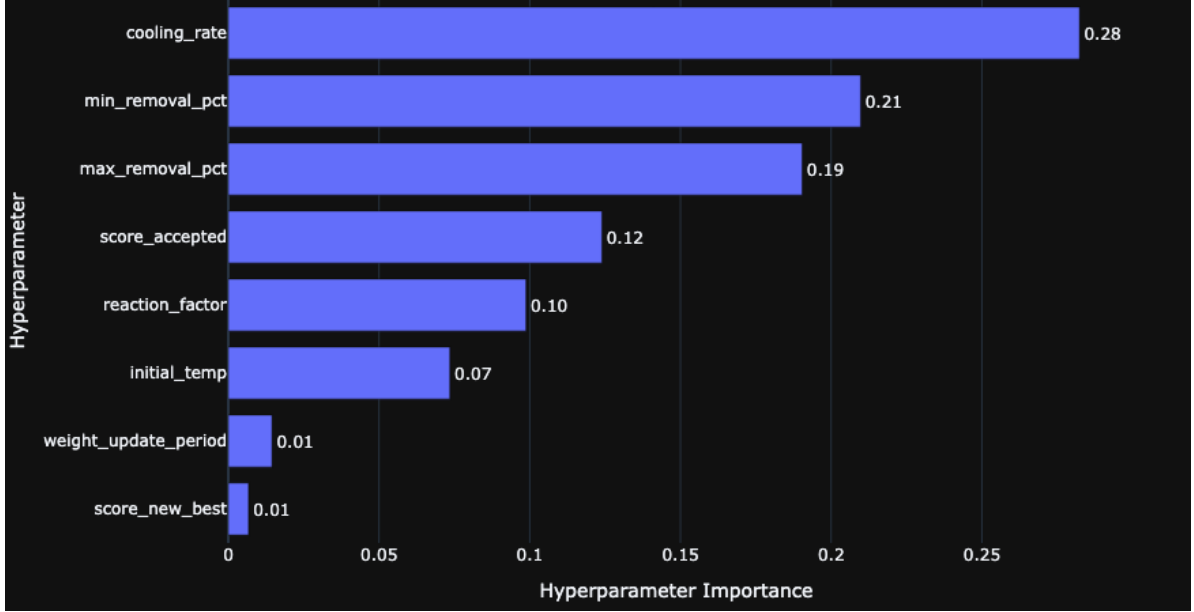


Figure 18: ALNS Parameter Importance for n=500

Instance Size n=1000: The tuning process has not converged within 2 hours so I had to interrupt it. In theory the runtime is defined by the timeout, so this was not supposed to happen but after looking at the dashboards provided by Optuna it seemed like the process got completely stuck which could happen due to deadlock in the sqlite database used for storing and visualizing results, as this database is not very good at handling concurrency. The framework of loading ALNS configs was adjusted to automatically discover the tuned parameters configuration of the closest instance size, thus for instance size 1000 the parameters tuned for instance size 500 will be used.

5.3 Impact of Parameter Tuning: ALNS Default vs ALNS Tuned

To quantify the value of automated parameter tuning, this experiment compares ALNS using default parameters against ALNS with Optuna-tuned instance-size-specific parameters. Both variants use identical destroy and repair operators, initial construction heuristic (`FlexiblePickupAndDropoffConstructionHeuristic`), and time budgets (30s for n=50, 60s for larger instances). The only difference is the parameter configuration: default values from Table 3 versus tuned values from Section 5.2.

Figure 26 presents the comparison across all instance sizes. The results of the comparison of default and tuned ALNS variants are very inconclusive. This is reflected in almost all recorded statistics - the total difference between objective values across all sizes expressed in relative percentage shows that the tuned configuration is better only by 0.1228%. In terms of total wins it's 74 for default vs 73 for the tuned and 3 ties - again very close. Looking at the p-value the only instance size where the H_0 can be confirmed is $n = 100$ in favor of the default configuration. It is hard to tell whether this is just a coincidence or there is a hidden pattern behind this instance size that makes the default hyperparameters produce better results. Bottom line is that tuning hasn't improved the parameters that were chosen by hand so there is no clear dominance of the tuned variant, although it starts slightly outperforming the default one for instance sizes 500 and 1000. The most possible explanation for that is limited tuning runtime due to high computational demand. The fine-tuning with Optuna was executed over 50 trials (combinations of randomly sampled parameters) which is probably not enough to learn patterns in large neighborhoods. Another limitation is the timeout that was set to 60 seconds and limited the ALNS exploration procedure in time for both variants. Given more time and resources it would be interesting to examine whether larger timeouts and permitted iterations let the ALNS algorithm improve the results significantly. Based on this comparison it can be assumed that any trends derived in the previous section were rather coincidental than consistent.

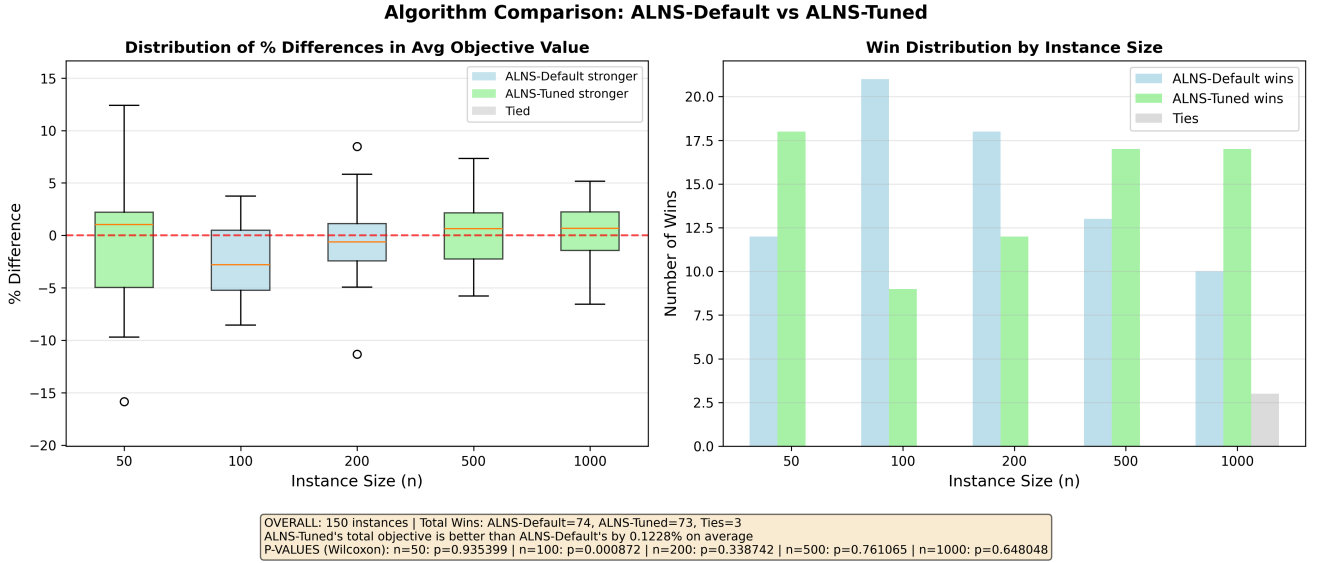


Figure 19: ALNS Default vs ALNS Tuned: Objective Value Comparison Across Instance Sizes

5.4 Simulated Annealing Tuning Results

Since tuning for the first assignment was done manually and with limited number of configurations I decided to fine-tune the simulated annealing solution using Optuna. First, the improved greedy construction was used as the initialization step of simulated annealing to help it start from a better solution. Then, the following parameters and ranges were fine-tuned on instances of sizes 50, 100, 200, 500 and 1000:

Table 7: SA Tuning Parameter Search Space

Parameter	Type	Search Range	Default
<code>initial_temperature</code>	Float	[10.0, 500.0]	100.0
<code>cooling_rate</code>	Float	[0.85, 0.9999]	0.95
<code>equilibrium_iterations</code>	Integer	[100, 1000]	500
<code>rcl_length</code>	Integer	[1, 20]	3

Similar to ALNS tuning, Optuna was executed with 50 trials using TPE sampler with seed 42 for reproducibility. The objective function minimized the average objective value across 5 training instances per size. Time budgets matched ALNS: 30 seconds for $n=50$ instances and 60 seconds for larger sizes ($n=100, 200, 500, 1000$).

5.4.1 Optimization Convergence and Parameter Importance

The following figures visualize the Optuna optimization process for Simulated Annealing across different instance sizes.

Instance Size $n=100$:

Figure 19 shows the optimization convergence over 50 trials for instance size 100. The Bayesian optimization discovers improvements throughout the trials, with the best objective appearing late. However, there is a plateau between ~ 6 -32 trials and only after that the fine-tuning procedure seems to have found a tendency in the hyper-parameters consistently improving the objective value in the last 10 trials.

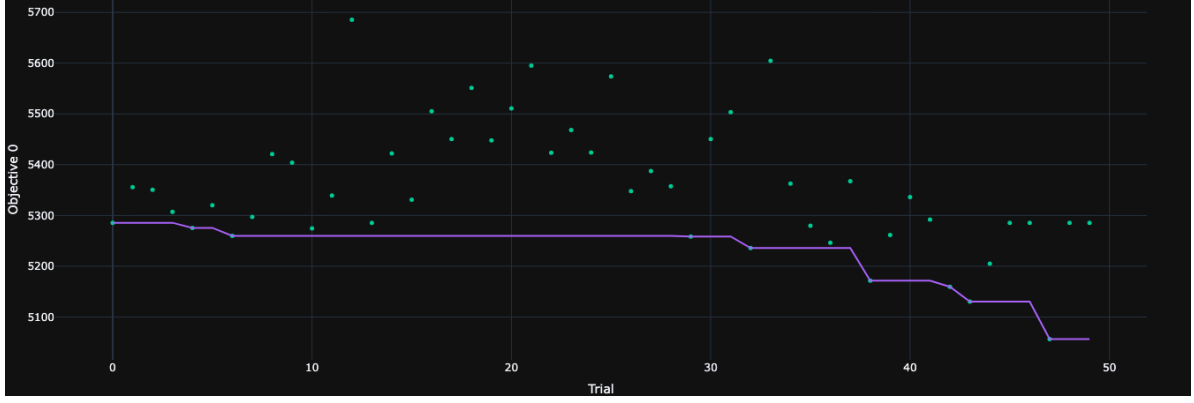
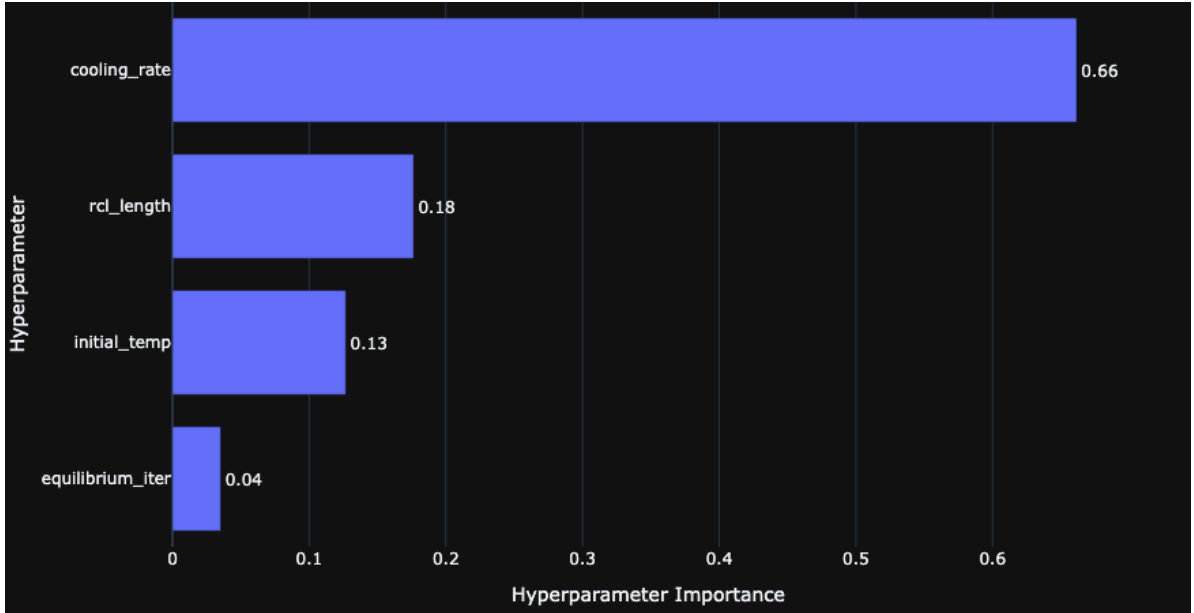
Figure 20: SA Tuning Convergence for $n=100$ (50 trials, 60s budget)

Figure 20 displays parameter importance for instance size 100. The outstanding parameter is `cooling_rate` weighing 0.66. The initial value of `cooling_rate` was 0.85 and the final one after 50 trials is 0.90. This is an indication that even for small instance size slower cooling is beneficial. However, initial temperature has dropped from initial 474 to final 147 which can explain the strong correlation between initial temperature and cooling rate that the Optuna was exploring instead of fixing them at certain ratio and trying to explore the combined effect with other parameters. To verify this hypothesis I extracted another visualization (see Figure 21) that captures correlation between pairs of parameters in respect to the objective value. Darker blue represents better objective value and we can see a strip along `cooling_rate` = 0.9. The surprising part here is that it seems that initial temperature has low impact on final result for this fixed cooling rate. One possible explanation could be that the neighborhood move is not very diverse so that larger or lower values of temperature throughout the execution do not result in significant variance of the objective value.

Figure 21: SA Parameter Importance for $n=100$

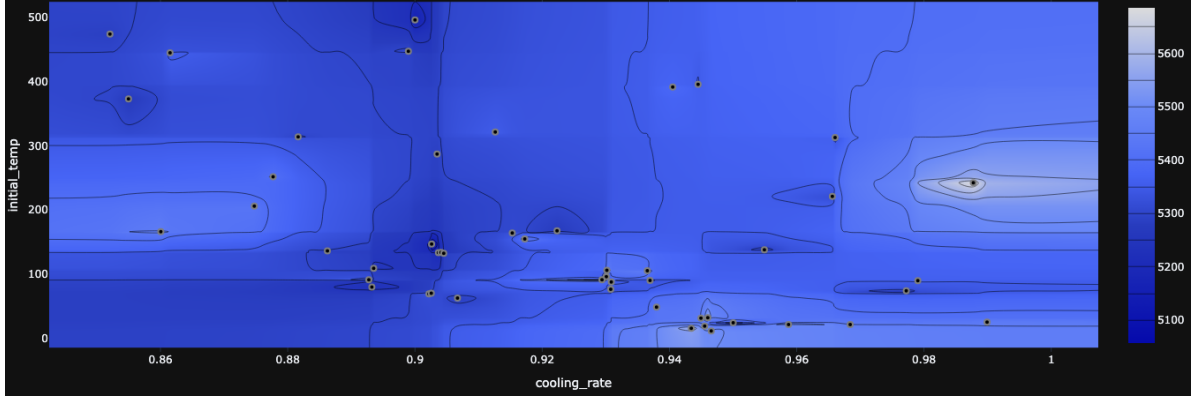


Figure 22: SA Parameter Correlation for n=100

Instance Size n=500:

Figure 22 shows the optimization convergence for instance size 500. There is a significant drop (almost 5000) of objective value in the beginning around 2nd trial, after which the objective value stays more or less the same indicating a local optima that cannot be escaped. The possible explanation is that the underlying randomized construction that managed to produce good results at this round. This is confirmed by Figure 23 which shows that the `rc1_length` was identified as the most impactful parameter (0.65). This parameter is the only one that affect the randomized construction and it is isolated from the rest.

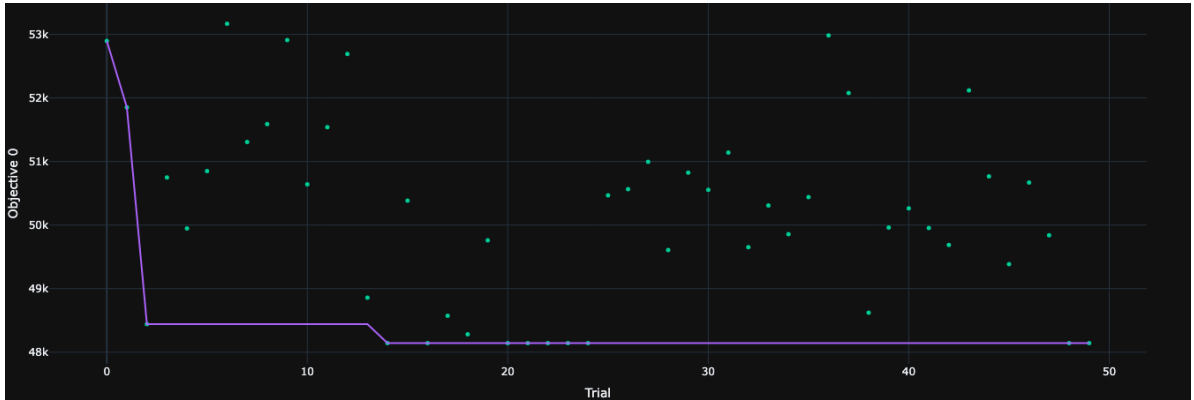


Figure 23: SA Tuning Convergence for n=500 (50 trials, 60s budget)

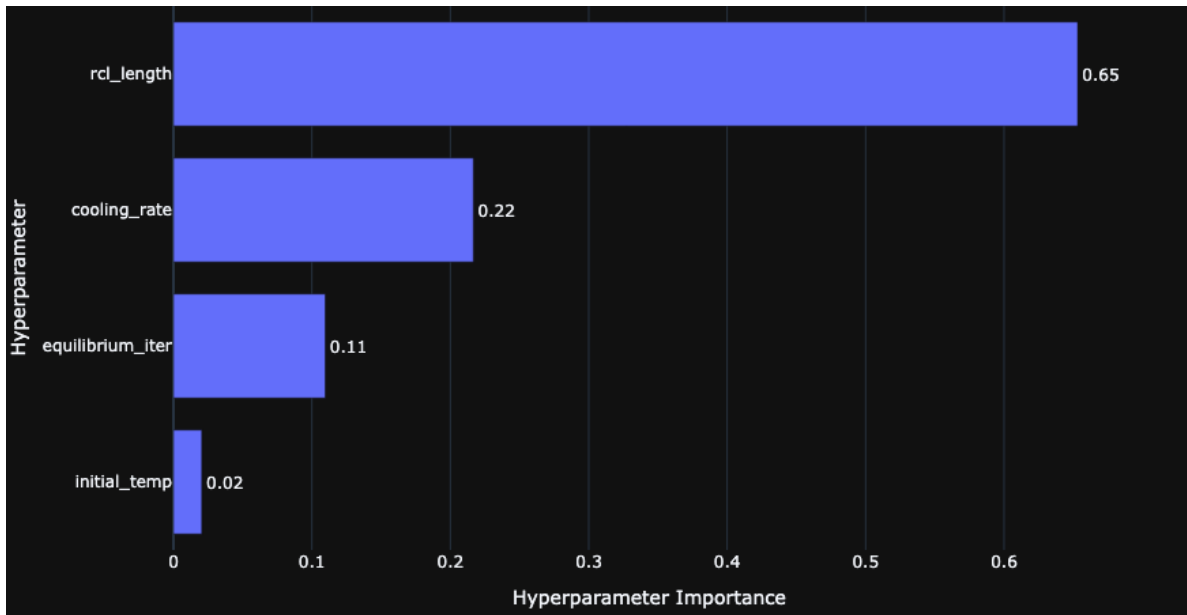


Figure 24: SA Parameter Importance for n=500

5.5 Instance Size Scalability

The `FlexiblePickupAndDropoff` construction heuristic scales reasonably well with instance size, with runtime growing polynomially as it evaluates insertion positions for each request. For instances up to size 1000, construction completes within 20-30 seconds, making it suitable as an initial solution generator for all tested instance sizes.

For ALNS, runtime is primarily controlled by stopping criteria: `max_iterations`, `max_time_seconds`, or `max_iterations_without_improvement`. While these parameters theoretically allow solving arbitrarily large instances by adjusting time budgets, practical scalability considerations arise from two factors. First, individual destroy and repair operations become more computationally expensive as instance size grows—evaluating feasible insertion positions and calculating objective values for larger solutions requires more iterations through longer routes. Second, the search space expands exponentially with instance size, requiring more iterations to achieve comparable solution quality. For the tuning experiments, I used 30-second timeouts for instances of size 50 and 60-second timeouts for sizes 100-1000, acknowledging that larger instances benefit from longer exploration time. The tuned parameters from smaller instances (n=50) may not transfer optimally to larger instances, as the balance between exploration and exploitation shifts with search space size. Based on conducted experiments, instances up to size 1000 can be solved with reasonable quality within 5-minute time budgets, while larger instances would require proportionally longer runtimes or instance-specific parameter tuning to maintain solution quality.

6 Statistical Comparison

6.1 Experimental Setup

All experiments were conducted on the test instances provided on the course website, on instance sizes 50, 100, 200, 500, and 1000 requests. For each instance size, all 30 available test instances were evaluated to ensure robust statistical analysis. The experimental evaluation focuses on comparing the two best approaches from each assignment: tuned Simulated Annealing and tuned ALNS.

The comparison methodology again leverages the `algorithm_comparison.py` framework developed for Assignment 1 and enhanced throughout Assignment 2, which provides:

- **Pairwise Comparison:** Each algorithm is run on identical test instances, enabling direct comparison of solution quality

- **Statistical Testing:** Wilcoxon signed-rank test is applied to determine whether observed differences are statistically significant ($p < 0.05$)
- **Performance Metrics:** Mean objective value, standard deviation, win/loss/tie counts, and construction time are recorded
- **Visualization:** Comparison plots show objective value trends across instance sizes, and histograms visualize the distribution of objective value differences between algorithms to indicate whether p-test is applicable

Both metaheuristics use `FlexiblePickupAndDropoffConstructionHeuristic` for initialization, ensuring that any performance differences stem from the metaheuristic search rather than initial solution quality. Time budgets were set consistently with the tuning phase: 30 seconds for $n=50$ instances and 60 seconds for all larger instances ($n=100, 200, 500, 1000$).

Each algorithm uses its instance-size-specific tuned parameters obtained from Optuna optimization (Section 5.2 for ALNS and Section 5.3 for SA), allowing fair comparison between optimally configured approaches rather than default parameter settings.

6.2 Comparison: Best from Assignment 1 vs. Best from Assignment 2

Figure 24 presents the comprehensive comparison between tuned Simulated Annealing and tuned ALNS across all instance sizes. The ALNS clearly dominates with total average improvement of 10% across all evaluated instance sizes. This result is not surprising given that the neighborhood structure that ALNS is exploring is much richer (larger) than that of SA. It could be interesting to further investigate different neighborhoods for the Simulated Annealing and maybe enhance it by choosing them from a distribution, although this is similar to what ALNS is doing. The Wilcoxon test shows clear domination of ALNS for all instance size even the ones with outliers where SA managed to find a much better local optima. It is worth noting that for larger instances (500 and 1000) the improvement of ALNS over SA is not as pronounced as for smaller instances. This could be explained by limited computational power, ALNS needs linearly more time and iterations to efficiently explore the large neighborhoods, while for SA there is no learning process meaning it can find decent results in fixed number of steps regardless of the instance size.

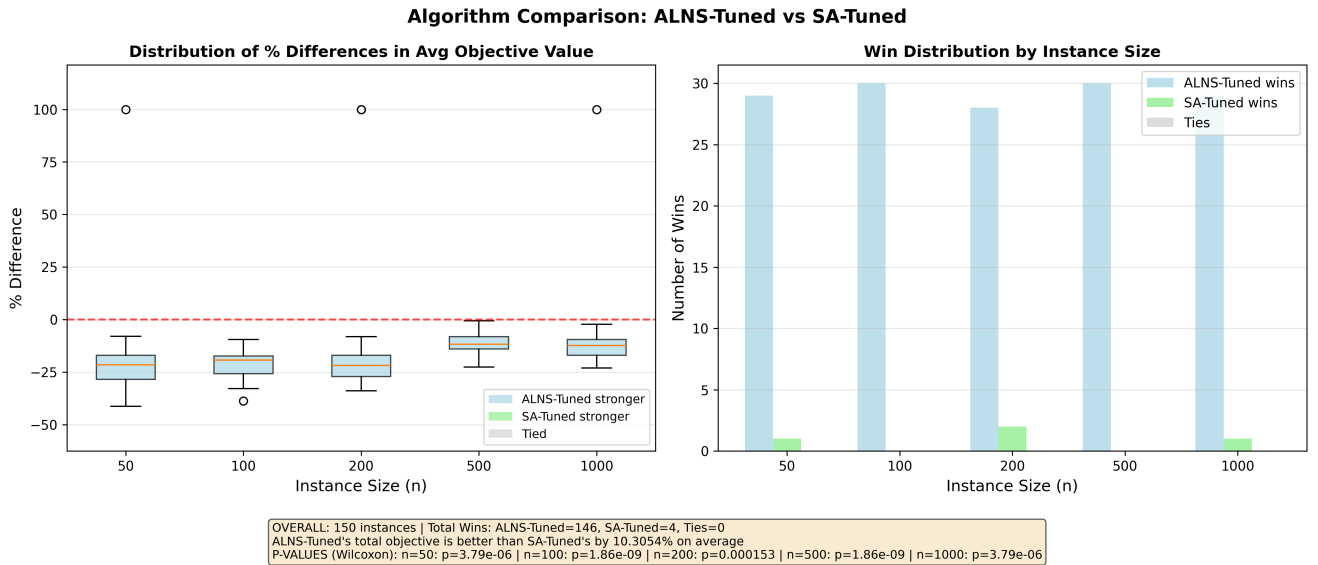


Figure 25: Tuned ALNS vs Tuned SA: Objective Value Comparison Across Instance Sizes

To better understand the distribution of performance differences and validate the applicability of the Wilcoxon signed-rank test, Figure 25 shows histograms of objective value differences (SA minus ALNS) for each instance size. We can see that the distribution of differences is not normal for most of the

instance sizes. In most cases there are long right tails and presence of outliers, making parametric tests like t-test inappropriate. The Wilcoxon signed-rank test is correctly chosen as it makes no distributional assumptions and is robust to outliers. Interestingly, $n = 500$ shows the most symmetric, almost normal distribution of differences, suggesting that at this scale the patterns of the algorithm behavior are the most predictable for both approaches.

While ALNS dominates on average, SA occasionally finds solutions comparable to or even better than ALNS on specific instances. These outliers indicate that SA’s performance variance is higher—it sometimes gets “lucky” with neighborhoods moves but lacks ALNS’s systematic exploration capability to consistently achieve high-quality solutions. This means ALNS offers more predictable performance and is more generalizable.

Instance Size Effects on Performance Gap: The distribution characteristics shift as instance size grows. For smaller instances ($n=50, 100$), the performance gap is larger and more variable. As instances grow to $n=500$ and $n=1000$, the distributions become more concentrated around smaller differences, suggesting that the relative advantage of ALNS’s rich neighborhood structure diminishes when computational budget becomes the limiting factor. This convergence implies that for very large instances under tight time constraints, the choice between SA and ALNS may matter less than for medium-sized problems where ALNS’s superior exploration can be fully leveraged.

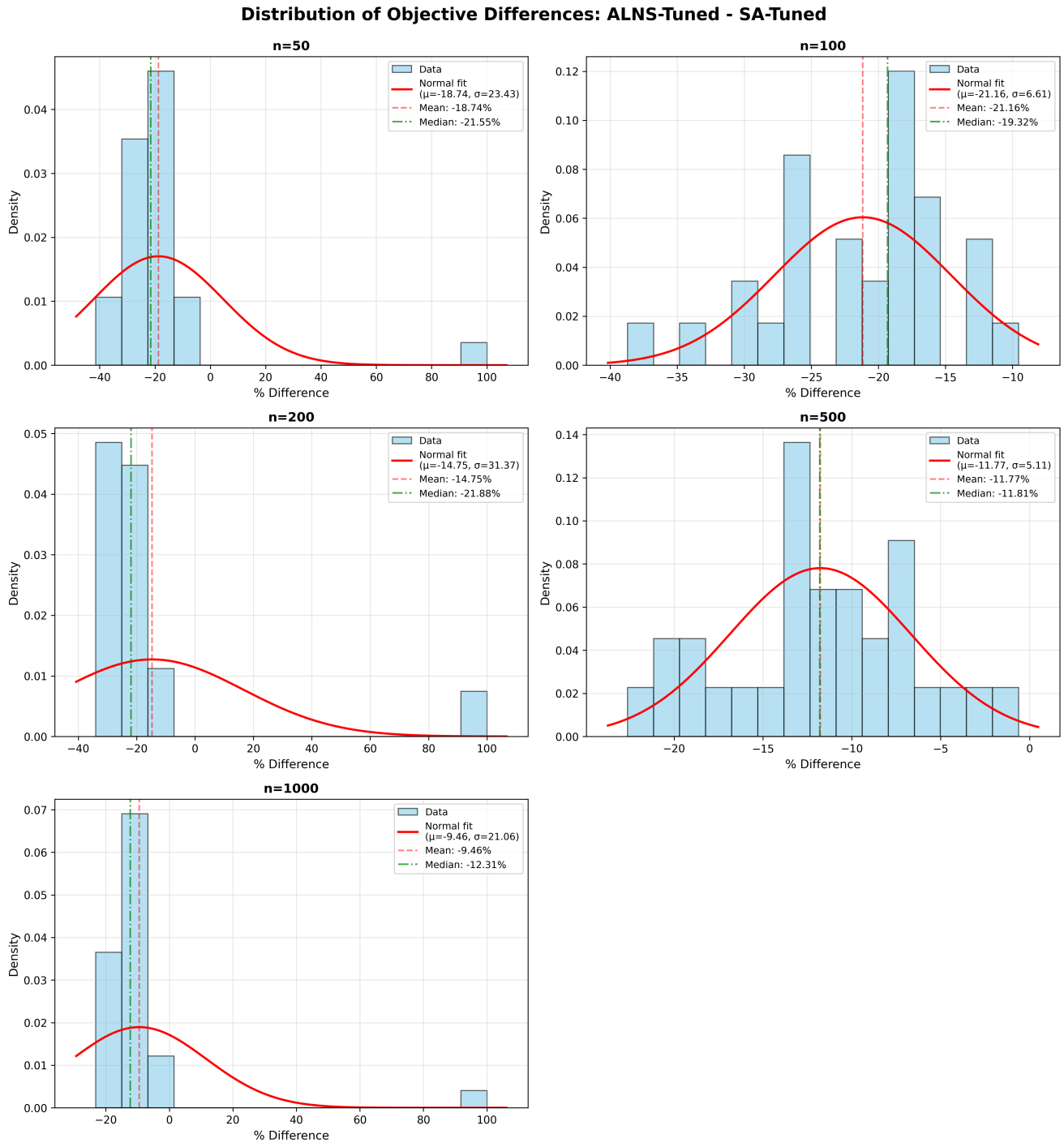


Figure 26: Distribution of Objective Value Differences (SA - ALNS) Across Instance Sizes

7 Reflection

7.1 Key Learnings

The most valuable aspect was building reusable infrastructure early. The `algorithm_comparison.py` framework from Assignment 1 made systematic evaluation across algorithms and instance sizes straightforward, with consistent statistical testing built in. Optuna eliminated manual parameter tuning completely—it found better configurations than my intuition-based defaults, and I could just let it run overnight.

Starting with better construction heuristics before jumping into metaheuristics was the right thing to

do. The FlexiblePickupDropoff improvement gave strong initial solutions that made metaheuristics much more effective. For practical applications, construction quality might matter as much as metaheuristic sophistication.

Visualization was critical for both understanding and debugging. Plotting different angles of the data (objective comparisons, fairness distributions, operator behavior) revealed insights that tables of numbers never would. Multiple times I caught bugs because plots looked suspicious—plotting became my main debugging tool.

The main challenges were all about infinite choices and limited time. Every decision spawned countless possibilities: which fairness metric, which operators, which parameter ranges. With 3×3 operators, 8 parameters, and 5 instance sizes, exhaustive exploration was impossible. I had to make pragmatic choices without certainty they were optimal.

Runtime was the other major constraint. Testing on large instances took 30-60 minutes per run, making rapid iteration impossible during development. This pushed most work to smaller instances, with uncertainty about whether findings would generalize.

As algorithms got more sophisticated (Greedy $\rightarrow$ SA $\rightarrow$ ALNS with adaptive weights $\rightarrow$ tuned ALNS), they became more effective but also more black-boxy. Understanding *why* something worked became harder, making improvements difficult compared to just trying things.

The provided instances seemed relatively balanced in distribution and capacity ratios. This raised questions about robustness: would these algorithms handle pathological cases like highly clustered requests or extreme capacity constraints? Performance on "normal" instances might not predict behavior on truly difficult problem structures.

7.2 Algorithm Design Insights

The fundamental challenge is balancing distance and fairness—they pull in opposite directions. Minimizing distance concentrates requests on efficient routes, while fairness demands balanced distribution. No algorithm perfectly reconciled both; the penalty weight γ just determines where you fall on the tradeoff curve.

The fairness investigation revealed something counterintuitive: operator diversity matters more than operator sophistication. Simple combinations (Random + WorstCost) often outperformed sophisticated operators explicitly designed for fairness. Having multiple operators prevents premature convergence and lets the adaptive mechanism learn what works.

This creates an evaluation dilemma: should operators overlap in functionality or be completely distinct? Do three removal operators provide more benefit than two? How do you evaluate operator quality when one operator might enable others to work better? With n operators yielding $2^n - 1$ combinations, exhaustive testing is infeasible, leaving much to educated guessing.

Parameters tuned on small instances ($n=50$) likely don't transfer well to large instances ($n=1000$) because the exploration-exploitation balance shifts with search space size. Instance-specific tuning would be ideal but computationally expensive.

7.3 Future Improvements

The most obvious improvement would be tracking operator runtime in ALNS and penalizing slow operators in the adaptive weights. Currently expensive operators reduce total iteration counts without penalty.

For evaluation, generating synthetic instances with extreme characteristics (tight clusters, outliers, capacity constraints) would test robustness beyond the provided instances. Systematically evaluating operators would reveal which ones actually matter versus which are redundant.

Parameter tuning could be improved by using transfer learning—initialize large instance tuning with small instance results rather than starting from scratch. More trials (100-200 instead of 50) might find better configurations, though with diminishing returns.

Despite limitations, the assignment showed that well-designed metaheuristics with automated tuning can produce high-quality solutions to complex problems within practical time.